

ECE60827: CUDA Assignment 3

Name: Nikhil Reddy Mummadi

GitHub username: mniksr786

Abstract

The CUDA kernels for convolutional layer implementation and general matrix multiplication (GEMM) were written and performance results for different input parameters are discussed.

Part A: Convolutional Layers (CONV)

CUDA Implementation:

All the threads inside each thread block collectively load the filter weights into the shared memory using strided access patterns, after which one thread per block iterates over all filter elements to perform the convolution sum and activation. Different batches run independently across a block index to process multiple inputs in parallel. The use of shared memory reduces the global memory accesses to improve the program latency.

Results:

The CUDA code was tested for convolution on five AlexNet layers whose parameters are provided in the assignment, and their performance was compared against the CPU implementation. As expected, the GPU runtime was consistently lower than the CPU runtime as shown in figure 1. The CPU runtime scales up aggressively for larger conv layers unlike the GPU runtime. The conv layer 3 has the maximum number of elements in its input tensors whose CPU runtime is ~6.5x larger than its corresponding GPU runtime, proving that the CUDA implementation is the favorable choice for conv layers with large input tensors.

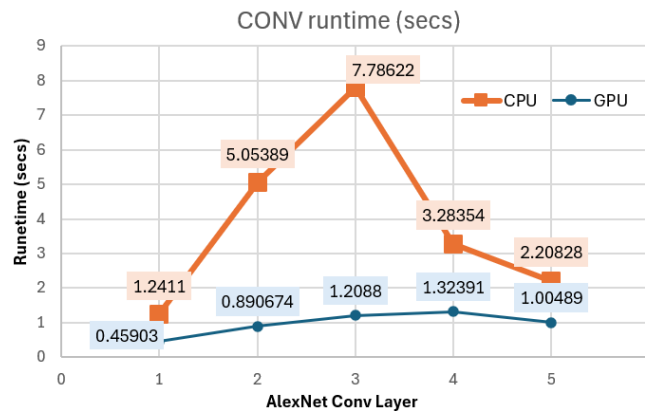


Figure 1: CONV runtime (secs) for AlexNet layers (CPU vs GPU)

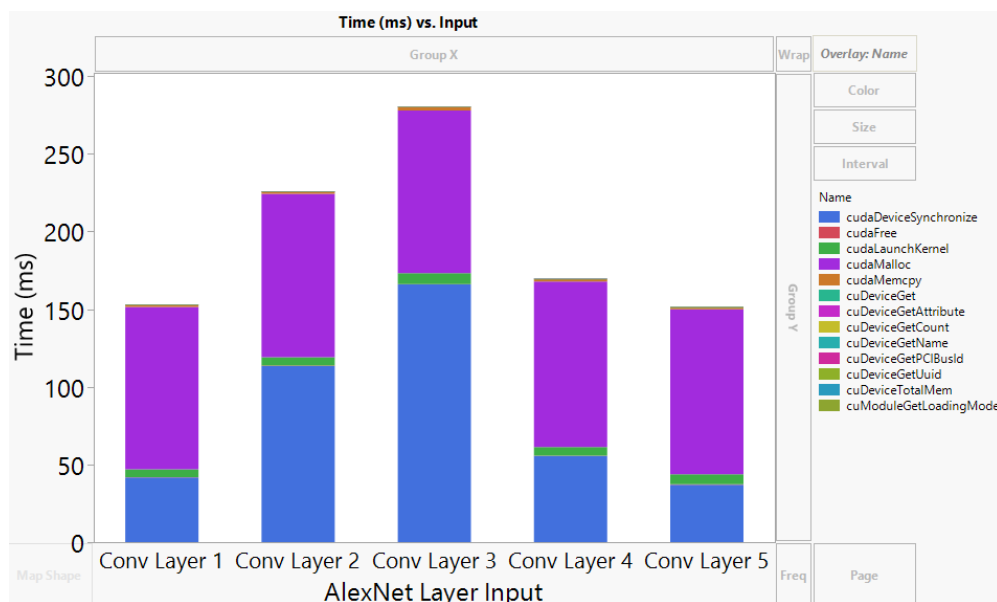


Figure 2: GPU CUDA API execution time (ms) for AlexNet Layer CONV

The CUDA APIs `cudaDeviceSynchronize` followed by `cudaMalloc` respectively consume majority of the program runtime as shown in figure 2. GPU memory allocation can be slow due to device-global memory management overhead due to which `cudaMalloc` is the second most called API in the program.

Part B: General Matrix Multiply (GEMM)

CUDA Implementation:

Each thread computes one element of the output matrix. The kernel uses shared memory tiling where the shared memory is statically allocated so that the compiler can optimize memory access patterns. There is padding in the shared memory allocation to minimize bank conflicts. CUDA streams are used here for asynchronous data transfers that can overlap with computations to maximize efficiency.

Results:

The GEMM code for CPU and GPU was tested on four input sizes of MNK. The input matrix A has dimensions {batchSize x 4096} (MxK) and the B matrix has dimensions {4096x4096} (KxN) whose values are provided in table 1. The input M corresponds to the batch size. The CUDA code was tested using CUDA memory copy and unified memory, where the latency for unified memory was only slightly higher than CUDA memory copy (discussed later). For smaller matrices, the CPU and GPU runtime are similar as expected. Since GEMM is mainly a compute bound workload, the parallelization benefit is obtained for large matrices like in inputs 3 and 4, where the GPU runtime was 5-9x lower than the CPU runtime as shown in table 2.

Cases	Tensor Inputs			Program runtime (secs)		
	M	K	N	CPU	GPU (CUDA mem cpy)	GPU (Unified Memory)
Input 1	3	4096	4096	12.2842	12.1599	12.6822
Input 2	256	4096	4096	43.3887	13.1165	13.1067
Input 3	512	4096	4096	73.309	13.5064	13.9804
Input 4	1024	4096	4096	135.027	15.1888	15.2283

Table 1: CPU vs GPU runtime for GEMM with various input matrices

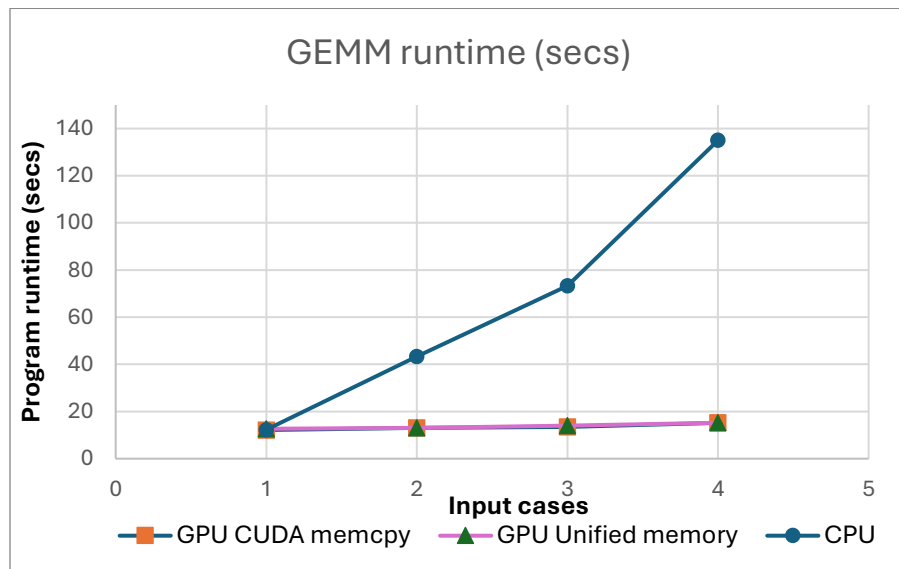


Figure 3: GEMM runtime (secs) for CPU vs GPU unified memory and CUDA memory copy

In unified memory, there are several data transfers between host and device along with multiple GPU page faults that result in page migrations. These data transfers and page faults add to the overall program latency. The stats in figure 4 indicate that gpu page faults incurred most of the latency overhead in unified memory.



Figure 4: Time (ms) and count of data transfers and page faults in unified memory mode

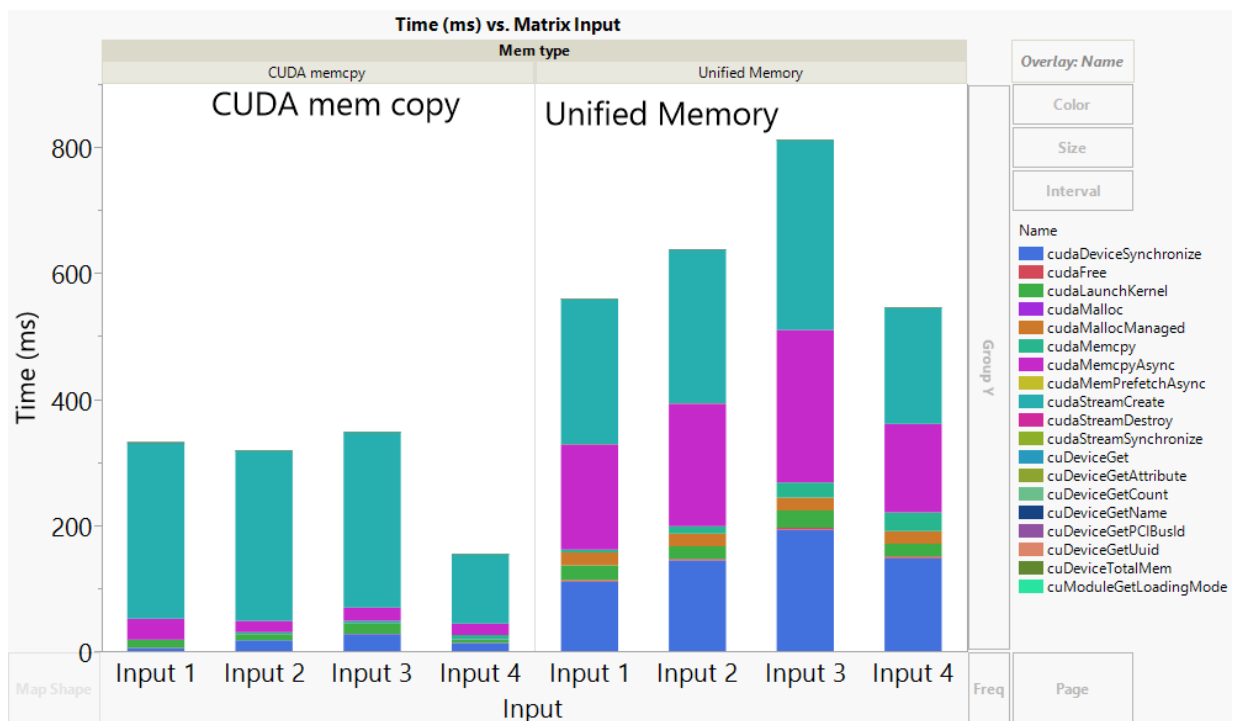


Figure 5: GPU CUDA API execution time (ms) for GEMM using CUDA memcopy and unified memory

When using CUDA memory copies, the API `cudaStreamCreate` takes majority of program runtime. But when operating with unified memory, the APIs `cudaMemcpyAsync` and `cudaDeviceSynchronize` contribute significantly to the overall runtime as synchronization time increases due to frequent page migrations and on-demand data transfers between host and device. Thus, using CUDA memory copies is preferable over unified memory to avoid page faults and reduce overhead.